

从零构建现代深度学习框架(TinyDL-0.01) ATA对外

1, 为什么要写一个深度学习框架?

1.1, 学习拥抱AI

1.2, just for fun, 向linux-0.01 致敬

2, 与别人的有什么不同?

一, 整体架构

1, 深度学习通用架构是什么样的?

2, TinyDL的整体架构是什么样的?

二, 线性代数与张量运算

1, 基础的线性代数

2, CPU版本的简单实现

3, 为什么需要GPU

三, 计算图与自动微分

1, 数值与解析微分

1.1, 数值微分

1.2, 解析微分

2, 计算图

3, 反向传播

链式法则

4, func层的设计与实现

四, 神经网络与深度学习

1, 误差反向传播算法

2, 层和块的堆积

五, 机器学习与模型

1, 数据集

2, 损失函数

3, 优化器

六, 应用任务与小试

- 1, 直线和曲线的拟合
- 2, 螺旋数据分类问题
- 3, 手写数字分类问题
- 4, 通过RNN拟合cos曲线

七, 小结

本文主要以一个Java工程师视角, 阐述如何从零(无任何二三方依赖)构建一个极简(麻雀虽小五脏俱全)现代深度学习框架(类比AI的操作系统)。这里的背景部分主要回答两个经常会碰到的灵魂之问: 一, 为什么要做; 二, 你做和别人做有什么区别?

1, 为什么要写一个深度学习框架?

1.1, 学习拥抱AI

AI是大趋势, 是长赛道, 这次不一样。在双拐点下Javaer的困境需要转型适应这个变化。在之前的文章《关于AIGC在互动上的若干实践》的最后我讲了下感受:

近年来市场与技术环境出现了双拐点: 一方面消费互联网的发展已经完全进入存量博弈时代增量红利不再, 降本增效成为主旋律; 另一方面AI技术的最新发展取得重大突破, 在很多方面的效率和成本已经明显超越人类。新时代新形势下也迫使像我这样的Javaer不得不去转型拥抱新技术。

对于技术上有两条新方向: 一条以3D为代表的元宇宙相关技术, 例如区块链, XR等技术的转型; 另外一条是以大模型为代表的AI方向的转型。最近开始学习深度学习相关的东西, 于是作为Java码农为了能看懂CNN, DNN, RNN等网络模型, 现在还在恶补线性代数和微积分以及PyTorch。虽都说“人生苦短, 我用python”, 可是想通过看源代码, 来学习深度学习框架的实现来了解原理的Javaer, Python天马行空的风格, 确实让我觉得人生苦短(这里没有任何言论战之意)。

大物理学家费曼说, “凡我不能创造的, 我就不能理解”。我想他大概是对的吧, 于是我也想试一试。

1.2, just for fun, 向linux-0.01 致敬

1991年9月17日, 21岁的芬兰学生林纳斯在网上发布了开源操作系统[Linux-0.01](#), 代码整洁, 目录清晰。本着东施效颦的做法, 我也特意将版本号成0.01, [TinyDL-0.01 \(Tiny Deep Learning\)](#), 也算向linux-0.01 致敬了。分享给对AI有兴趣Javaer, 可以从底层工程角度了解深度学习的原理以及简单的实现, 有兴趣的小伙伴可以一起相互学习。

2，与别人的有什么不同？

深度学习框架有很多，比较多用python编写的(底层是C/C++) 例如 TensorFlow, PyTorch以及MXNet等。Java实现的深度学习框架比较知名的有两个：一个是[DeepLearning4J](#)由于Eclipse开源社区维护，还有一个[DJL](#)是AWS的开源的深度学习 Java 框架。

DeepLearning4J是一个全栈的实现，过于复杂庞大的技术栈（65%Java 69.7w行，24%C++，3.4% CUDA等）且依赖过多复杂科学计算的三方库，显然很难通过代码来学习。

DJL只是一套面向深度学习的Java高层次的接口，并没有任何真正的实现，最终是运行在TensorFlow或PyTorch深度学习引擎上的。

[TinyDL-0.01](#)如它的名字一样，是一个最小化实现的轻量级深度学习框架（Java实现）。相比与其他实现：1) 极简，基本上零三方依赖（为了不想引入JUnit，我有了不写单测的理由了）；2) 全栈，从最底层的张量运算到最上层的应用案例，3) 分层易扩展，每一层的实现都包括了核心概念和原理且层边界清晰。当然不足之处更明显：功能简陋性能较差。

TinyDL-0.01虽然只是一个toy级别的框架，但它尝试具有现代深度学习框架的基本特性（动态计算图，自动微分，多优化器，多类型网络层实现等），主打一个简单明了，主要用于入门学习只用。如果想通过看PyTorch等框架的代码来深度理解深度学习，那基本直接劝退。

一，整体架构

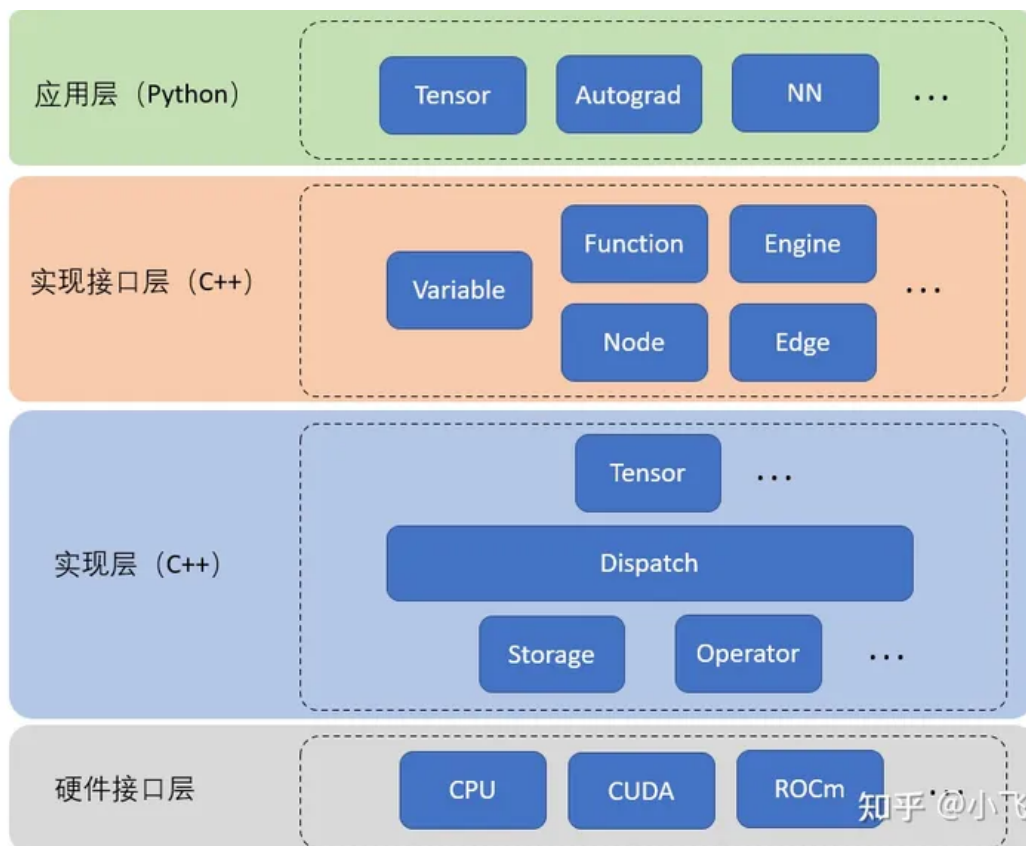
深度学习框架主要解决的是深度网络训练和推理的工程问题，包括多层神经网络组成的复杂性问题，大量的矩阵运算和并行计算的效率问题，以及支持在多个计算设备的扩展性问题等等。常用的深度学习框架包括：TensorFlow，由Google开发的开源框架；PyTorch，由Facebook开发的开源框架；MXNet，由Amazon开发的开源框架等等。经过多年的发展，它们的架构和功能都慢慢趋同，先看下一个现代深度学习框架的通用能力有哪些。

1，深度学习通用架构是什么样的？

先看下chatGPT是如何回答这个问题的：

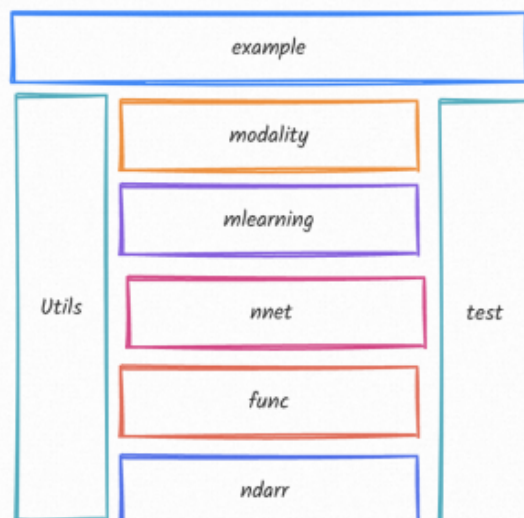
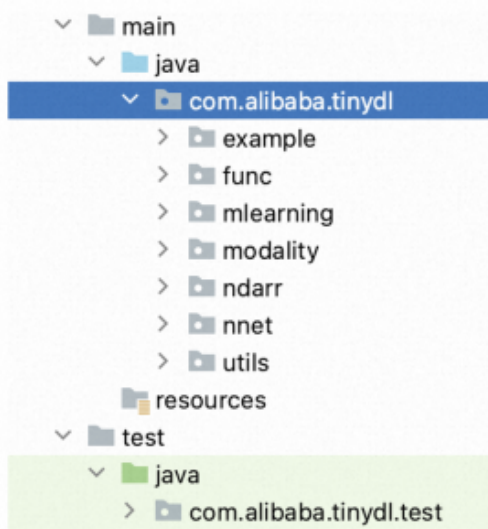


这里也具体参考当下最流行的深度学习框架PyTorch，大体分成四层（来自知乎）：



2, TinyDL的整体架构是什么样的?

TinyDL的秉承简洁分层清晰的原则，并参考了通用的分层逻辑，整体的结构如下：



从下至上保持严格的分层逻辑：

- 1, ndarr包：核心类 NdArray，底层线性代数的简单实现，目前只实现CPU版本，GPU版本需要依赖庞大的三方库。
- 2, func包：核心类Function与Variable 分别是抽象的数学函数与变量的抽象，用于在前向传播时自动构建计算图，实现自动微分功能，其中Variable对应PyTorch的tensor。
- 3, nnet包：核心类Layer与Block表示神经网络的层和块，任何复杂的深度网络都是依赖这些Layer与Block的堆叠而层。实现了一些常用的cnn层rnn层norm层以及encode与decode的seq2seq架构等等。
- 4, mlearning 包：机器学习的通用组件的表示，深度学习是机器学习的一个分支，对应更广泛的机器学习有一套通用的组件，包括数据集，损失函数，优化算法，训练器，推导器，效果评估器等。
- 5, modality 包：属于应用层的范畴，目前深度学习主要应用任务图形图像的视觉，自然语言处理以及强化学习三部分，暂时还没有相应的领域的实现，希望在0.02版中实现GPT-2等原型。
- 6, example包：一些简单的能跑通的例子，主要包括机器学习的分类和回归两类问题，有曲线的拟合，螺旋曲线的分类，手写数字的识别以及序列数据的预测。

接下来就从下至上，全栈式地简答串一下每层涉及的核心概念和简单实现。

二，线性代数与张量运算

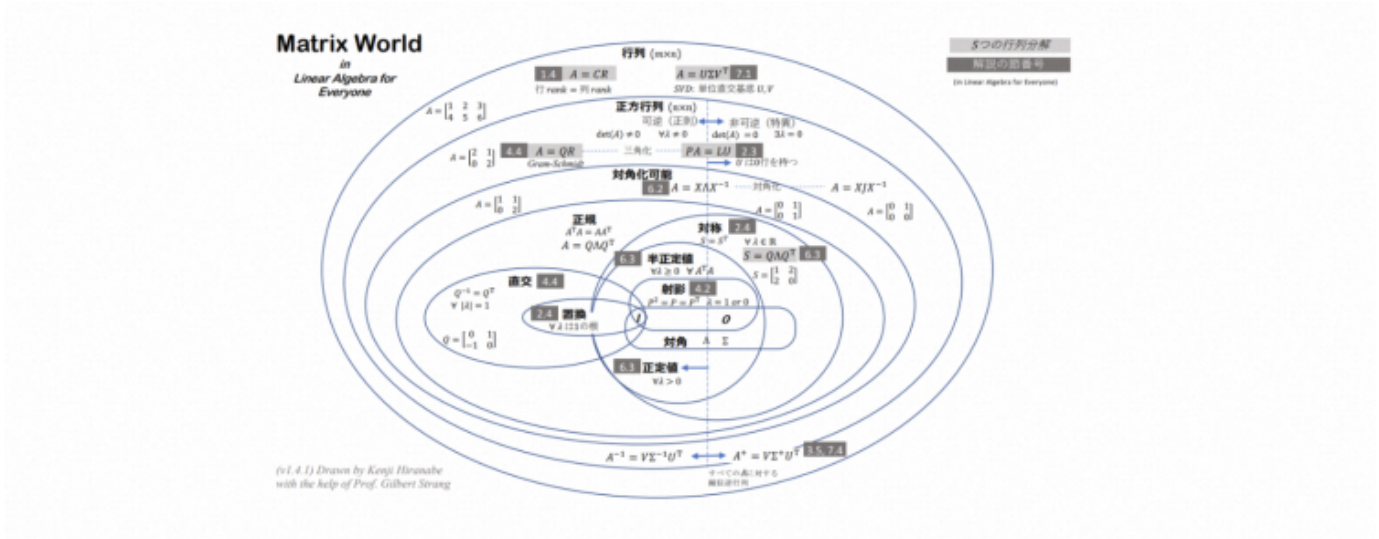
首先进入深度学习的第一层：张量操作层。张量（多维数组）的操作和计算是深度学习的基础，所有运算几乎都是基于张量的运算（单个数字称为标量，一维数称为向量，二维数组称为矩阵，三维以及以上

数组称为N维张量）。这些操作通常使用高效的数值计算库来实现，通过C/C++语言在特定的计算硬件上实现的，提供了各种张量操作，如矩阵乘法、卷积、池化等。

这一部分主要分三块，首先关于线性代数的一些基本知识，然后是基于CPU的最小化实现，最后对比为什么深度学习重依赖一种全新的计算范型GPU。

1，基础的线性代数

先上一张图来直接劝退下大家，虽然当初考研的时候，这些只算最基本的八股练习题：



然后是一些线性代数的常见概念：

- 1. 向量：向量是一个有大小和方向的量，在线性代数中，向量通常用一系列数表示。
- 2. 矩阵：矩阵是一个二维数组，由行和列组成，它可以用于表示线性方程组或者线性变换。
- 3. 向量空间：向量空间是由一组向量构成的集合，满足一些特定的性质，如封闭性、加法和数量乘法的结合性等。
- 4. 线性变换：线性变换是一种将一个向量空间映射到另一个向量空间的操作。它保留线性组合和共线关系。
- 5. 线性方程组：线性方程组是一组线性方程的集合，其中每个方程都满足变量的次数为1，并且具有线性关系。
- 6. 特征值和特征向量：在矩阵中，特征值是一个标量，特征向量是一个非零向量，满足矩阵与该向量的乘积等于特征值乘以该向量。
- 7. 内积和外积：内积是向量之间的一种运算，用于度量它们之间的夹角和长度，外积是向量之间的一种运算，用于生成一个新的向量，该向量垂直于原始向量。
- 8. 行列式：行列式是一个标量值，由一个方阵的元素按照特定的规则组合而成，它用于计算矩阵的逆、判断矩阵的奇偶性等。

有没有点头大？但是如果你看到[CPU版本的简单实现](#)，你也会瞬间觉得如此简单（目前只支持标量&向量&矩阵，暂不支持更高维度的张量）。

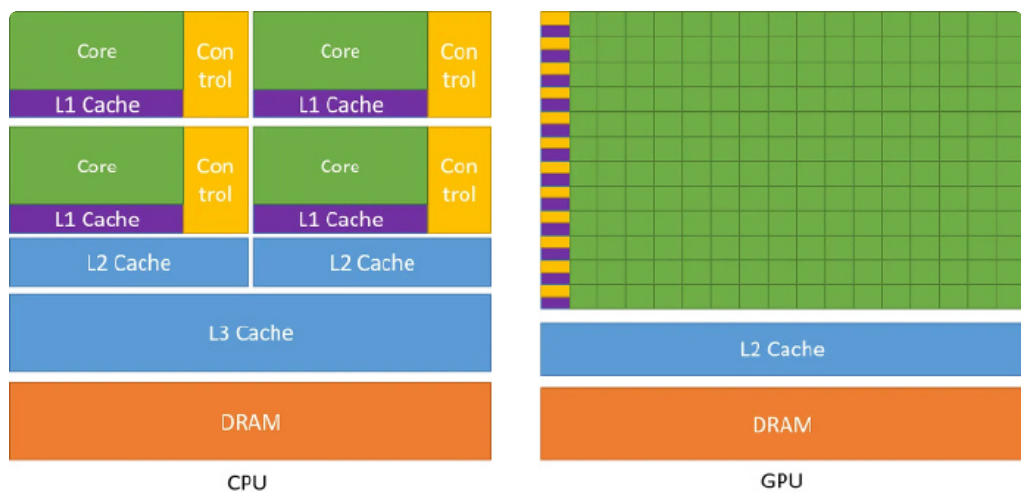
2，CPU版本的简单实现

```
1  /**
2   * 支持，1，标量；2.向量；3，矩阵，
3   * <p>
4   * 暂不支持，更高纬度的张量，更高维的通过Tensor来继承简单实现
5   */
6
7  public class NdArray {
8
9      protected Shape shape;
10
11     /**
12     * 真实存储数据，使用float32
13     */
14     private float[][] matrix;
15 }
16
17 /**
18 * 表示矩阵或向量的形状
19 */
20
21 public class Shape {
22     /**
23     * 表示多少行
24     */
25     public int row = 1;
26     /**
27     * 表示多少列
28     */
29     public int column = 1;
30
31     public int[] dimension;
32 }
```

其实都是一些对二维数组的操作：大致分三类，首先是一些简单初始化函数；其次是基本的四则运算加减乘除，还有一些运算会对矩阵改变形状，其中内积是最常的。

3，为什么需要GPU

通过上面在CPU上实现的的矩阵操作，特别是一个简单内积运算需要多层for循环嵌套，大家就知道，在CPU上这种为逻辑控制转移设计的架构，其实并不能很好地实现并行运算。而矩阵运算的行列其实是可以并行的，所以深度学习依赖的矩阵运算在CPU上是极其低效的。为了更直观地对比可以参考下图，相比于CPU，GPU的控制逻辑单元较弱（蓝色单元），但是具有大量的ALU(算术逻辑 绿色单元)。



大部分深度学习框架（如TensorFlow、PyTorch等）都提供了对GPU的支持，可以方便地利用GPU进行并行计算。随着近年比特币区块链以及今年chatGPT的爆发，GPU已经成为AI的基础设施，快速成为一种全新的主流计算范式。写到这里忍不住看了下英伟达的股价：这波淘金热，它应该属于卖铲子的。



三，计算图与自动微分

现在来到深度学习框架的第二层：func层，主要实现深度学习框架非常重要的特性，计算图与自动微分。

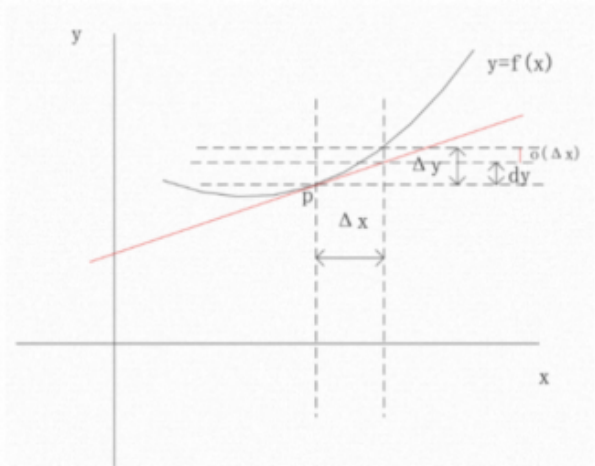
- 1) 计算图是一种图形化表示方式，用于描述计算过程中数据的流动和操作的依赖关系。在深度学习中，神经网络的前向传播和反向传播过程可以通过计算图来表示。
- 2) 自动微分是一种计算导数的技术，用于计算函数的导数或梯度。在深度学习中，反向传播算法就是一种自动微分的方法，用于计算神经网络中每个参数对于损失函数的梯度。

通过计算图和自动微分的结合，可以有效地计算复杂神经网络中大量参数的梯度，从而实现模型的训练和优化。

1，数值与解析微分

1.1，数值微分

导数是函数图像在某一点处的斜率，也就是纵坐标增量（ Δy ）和横坐标增量（ Δx ）在 $\Delta x \rightarrow 0$ 时的比值。微分是指函数图像在某一点处的切线在横坐标取得增量 Δx 以后，纵坐标取得的增量，一般表示为 dy 。



数值微分是一种用数值方法来近似计算函数的导数的方法，其目的是通过计算函数在某个点附近的有限差分来估计函数的导数值。求解使用比较多的是中心差分，通过近似计算函数在某个点的导数，使用函数在该点前后一个点的函数值来计算，公式如下： $f'(x) \approx (f(x + h) - f(x - h)) / (2h)$ 。其中，h是差分的步长，步长越小，计算结果越精确。数值微分是一种近似计算方法，计算结果与真实的导数值存在一定误差。

1.2，解析微分

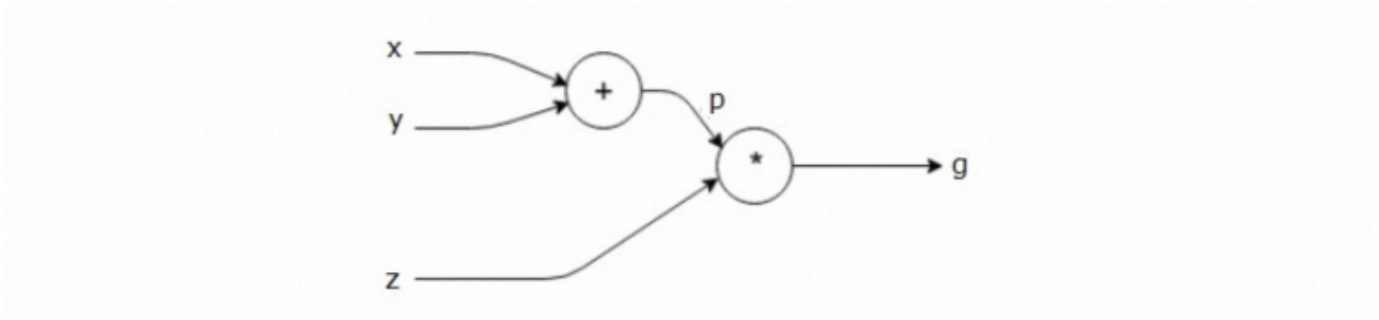
解析微分是微积分中的另一种方法，用于精确计算函数在某个点的导数值。它通过应用导数的定义和基本的微分规则来求解导数。可以根据函数的定义，确定函数表达式。例如，给定一个函数 $f(x)$ ，需要确

定它的表达式，如 $f(x) = x^2 + 2x + 1$ 。例如以下是一些常用函数的解析微分：

一阶导数		
编号	原函数	导函数
①	$y = c$	$y' = 0$
②	$y = a^x$	$y' = a^x \ln a$
③	$y = \log_a x$	$y' = \frac{1}{x \ln a}$
④	$y = \ln x$	$y' = \frac{1}{x}$
⑤	$y = x^n$	$y' = nx^{n-1}$
⑥	$y = \sqrt[n]{x}$	$y' = \frac{x^{-\frac{n-1}{n}}}{n}$
⑦	$y = \frac{1}{x^n}$	$y' = -\frac{n}{x^{n+1}}$
⑧	$y = \sin x$	$y' = \cos x$
⑨	$y = \cos x$	$y' = -\sin x$
⑩	$y = \tan x$	$y' = \frac{1}{\cos^2 x} = \sec^2 x$

2，计算图

计算图被定义为有向图，其中节点对应于数学运算， 计算图是表达和评估数学表达式的一种方式。例如， 有以下等式 $g=(x+y)*z$ ， 绘制上述等式的计算图。

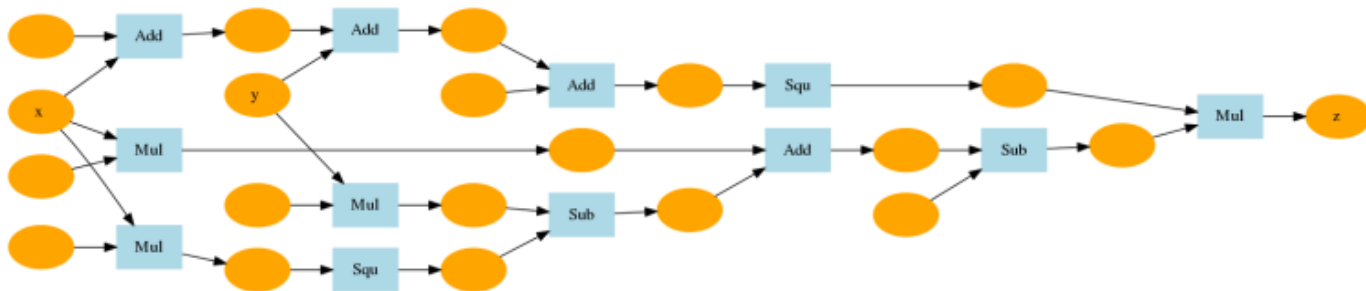


计算图具有加法节点（具有“+”符号的节点）和乘法节点， 其具有三个输入变量xyz以及一个输出g。

现在看下一个更复杂的函数：

$$f(x,y) = [1 + (x + y + 1)^2 * (19 - 14x + 3x^2 - 14y + 6xy + 3y^2)] * [30 + (2x - 3y)^2 * (18 - 32x + 12x^2 + 48y - 36xy + 27y^2)]$$

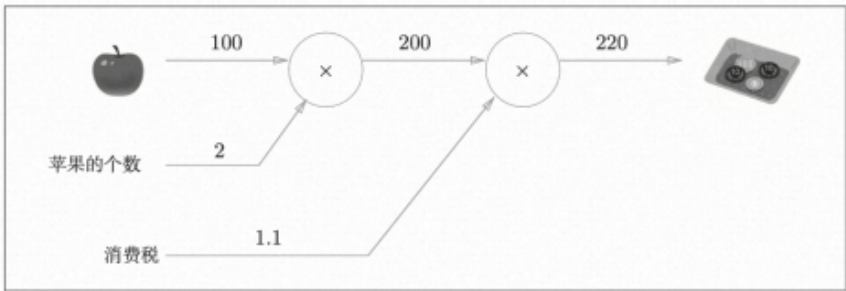
以下函数f(x,y)的计算图如下：



计算图的优点是能够清晰地表示复杂的函数计算过程，并且方便进行反向传播和参数更新。在深度学习中，计算图通常用于构建神经网络模型，其中每个节点表示神经网络的层或操作，边表示神经网络层之间的数据流。通过构建计算图，可以将复杂的函数计算过程分解为一系列简单的操作，利用反向传播算法计算每个节点的梯度，从而实现对模型参数的优化和训练。

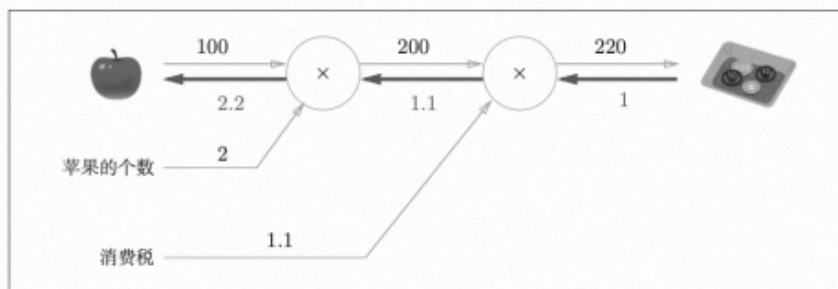
3，反向传播

网上找到一个比较形象的例子来说明反向传播：假设现在要购买水果，在我们日常的思维当中这是一件非常简单的事情，计算一下价格然后给钱就完事了，但实际上这个过程可以抽象成一张计算图，当中是包含了若干步骤的。比如下图，看到需要先计算苹果的价格乘上个数，再乘上消费税，最后得到的才是真正的支付额。



上面的过程叫正向传播，那反向传播也很好理解，字面意思是方向相反，但是实际上计算方式也略有不同。假设在上图买水果的例子当中，我们希望知道苹果对于最终价格的导数是多少，我们应该怎么计算？

正向传播是为了计算模型的最终结果，那么反向传播是为了什么？当然是为了得到模型当中每一个参数对于结果的影响系数，从而可以根据这个系数调整参数，使得模型的结果更好。很显然需要反向一层一层地往前计算，也就是一层一层地把系数乘起来，其实这里的系数就是每一层的导数。

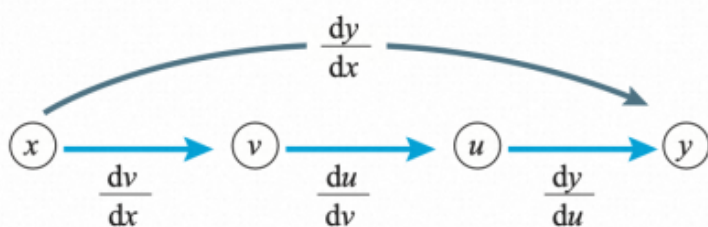


反向传播使用与正方向相反的箭头(粗线)表示，反向传播传递“局部导数”，将导数的值写在箭头的下方。在这个例子中，反向传播从右向左传递导数的值(1→1.1→2.2)。从这个结果中可知，“支付金额关于苹果的价格的导数”的值是 2.2。这意味着，如果苹果的价格上涨 1 日元，最终的支付金额会增加 2.2 日元（严格地讲，如果苹果的价格增加某个微小值，则最终的支付金额将增加那个微小值的2.2 倍）。

链式法则

链式法则是微积分中的一个重要定理，用于求复合函数的导数，偏导数是多元函数对其中一个变量的偏微分，链式法则同样适用于多元函数的偏导数。假设有两个函数： $y = f(u)$ 和 $u = g(x)$ ，其中 y 是 x 的函数。那么根据链式法则， y 对 x 的导数可以通过求 f 对 u 的导数和 g 对 x 的导数的乘积来计算。具体而言，链式法则可以表示为： $\frac{dy}{dx} = \frac{dy}{du} * \frac{du}{dx}$ 。要计算一个复杂函数的导数，采取的就是链式求导的方式。神经网络其实本质上就可以看成是一个复杂函数，就是对一个复杂函数求导。

$$\frac{dy}{dx} = \frac{dy}{du} \frac{du}{dv} \frac{dv}{dx}$$



三个函数的复合函数的链式法则。与两个变量的情形一样，可以像分数一样进行计算。

很难简单讲清楚，毕竟背后是大学通识课里最难的，大学第一年很多同学都挂在了这棵树上-高数，感兴趣的同学可以参阅《[深度学习的数学](#)》。

4，func层的设计与实现

func层最核心的类就是 Function以及Variable，对应于一个抽象的数学函数形式 $y=f(x)$ 中， x 和 y 表示 Variable 变量， $f()$ 表示一个函数Function。每个具体的function的实现需要实现两个方法：forward与

backward。

```
1  /**
2      * 函数的前向传播
3      *
4      * @param inputs
5      * @return
6      */
7  public abstract NdArray forward(NdArray... inputs);
8
9  /**
10     * 函数的后向传播，求导
11     *
12     * @param yGrad
13     * @return
14     */
15  public abstract List<NdArray> backward(NdArray yGrad);
```

例如一个Sigmoid的函数实现如下：

```
1  public class Sigmoid extends Function {
2      @Override
3      public NdArray forward(NdArray... inputs) {
4          return inputs[0].sigmoid();
5      }
6
7      @Override
8      public List<NdArray> backward(NdArray yGrad) {
9          NdArray y = getOutput().getValue();
10         return Collections.singletonList(yGrad.mul(y).mul(NdArray.ones(y.getShape()).sub(y)));
11     }
12
13     @Override
14     public int requireInputNum() {
15         return 1;
16     }
17 }
```

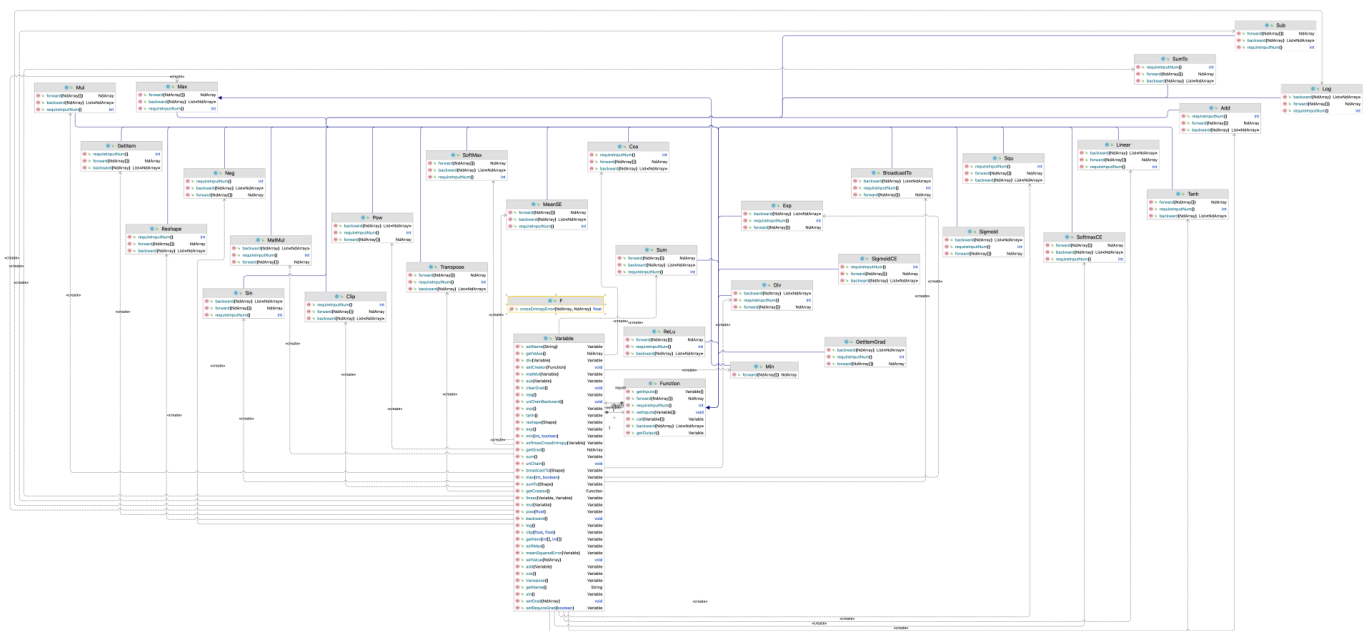
Variable 的实现如下，表示数学中的变量的抽象，其中backward 是反向传播的入口函数。

```

1  **
2  * 数学中的变量的抽象
3  */
4  public class Variable {
5      private String name;
6
7      private NdArray value;
8
9      /**
10     * 梯度
11     */
12     private NdArray grad;
13
14     /**
15     * 保持是函数对象生成了当前Variable
16     */
17     private Function creator;
18
19     private boolean requireGrad = true;
20
21     /**
22     * 变量的反向传播
23     */
24     public void backward() {}
25 }
26

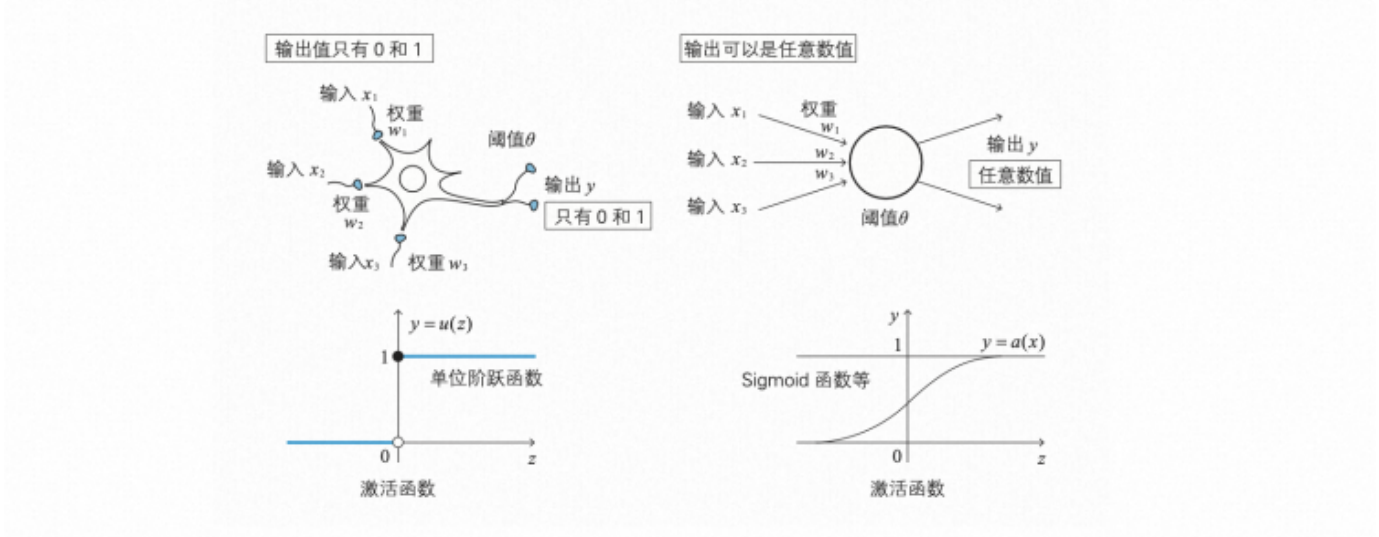
```

整个func层的实现如下：



四，神经网络与深度学习

神经网络由多个神经元组成的网络，在模拟生物神经系统的基础上进行信息处理和学习。神经网络的设计灵感来源于人类大脑中神经元之间的相互作用。在神经网络中，每个神经元接收来自其他神经元的输入，并根据输入的权重进行加权求和，然后将结果传递给激活函数进行处理并产生输出。神经网络的学习过程通常通过调整网络中神经元之间的连接权重来实现，使得网络能够根据输入数据进行预测和分类。如下图是神经元的生物和数学两种模型的表示。



神经网络是由多个节点组成的层次结构，每个节点通过加权和非线性激活函数的计算来处理输入数据，并将结果传递给下一层节点。而深度学习是在神经网络基础上使用多个隐藏层的深层神经网络。神经网络是深度学习的基础模型，而深度学习在神经网络的基础上引入了多层网络结构，可以自动学习更加抽象和高级的特征表示。

1，误差反向传播算法

2006年，Hinton等人提出了一种基于无监督学习的深度神经网络模型。模型的训练通过一种全新的方法来解决梯消失的问题，逐层训练打破了以往深度网络难以训练的困境，为深度学习的发展奠定了基础开创了深度学习。但是现代深度学习仍然是采用误差反向传播（backpropagation）算法进行训练的，主要原因有：一些新的激活函数的提出，正则化参数初始化等方法的改进，还有全网络的梯度下降训练的高效等等。

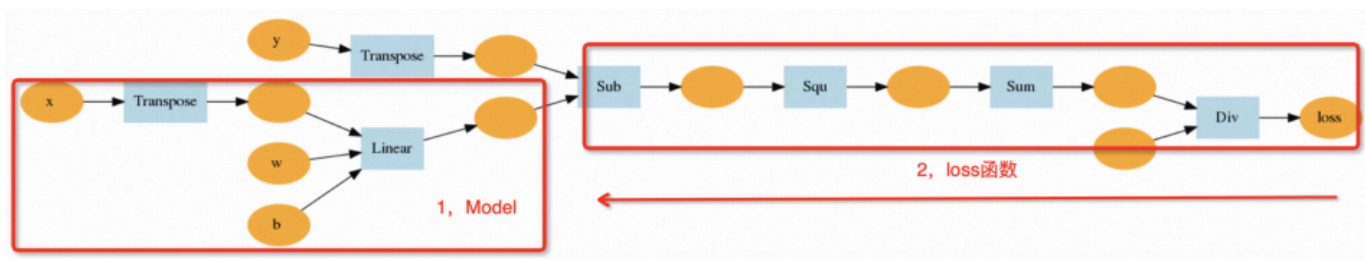
反向传播算法是训练神经网络的主要方法，基于梯度下降，通过计算损失函数对网络参数的梯度，然后按照梯度的反方向调整网络参数，从而使得网络的输出更接近于真实值。反向传播算法利用链式法则，将网络的输出与真实值之间的误差逐层传递回网络的输入层，计算每一层参数的梯度。具体如下：

1. 正向传播：将输入样本通过神经网络的前向计算过程，得到网络的输出值。
2. 计算损失函数：将网络的输出值与真实值之间的差异，作为损失函数的输入，计算网络预测值与真

实值之间的误差。

3. 反向传播：根据损失函数的值，逐层计算每个参数对损失函数的梯度。通过链式法则，将上一层的梯度乘以当前层的激活函数对输入的导数，得到当前层的梯度，并传递到前一层，逐层计算直到输入层。
4. 更新网络参数：使用梯度下降算法，按照梯度的反方向更新每个参数的值，使得损失函数逐渐减小。
5. 重复上述步骤，直至达到训练停止的条件，或者达到最大迭代次数。

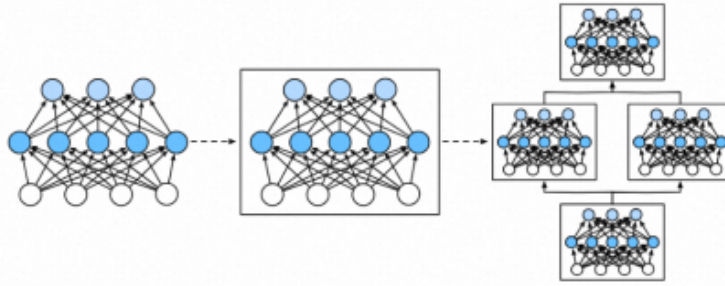
反向传播算法的核心思想，是通过计算每一层参数的梯度，逐层更新网络参数，从而使网络能够逼近真实值。



误差反向传播算法在深度神经网络中面临的一个常见挑战是梯度消失问题。为解决梯度消失问题，很多方法被提出：1) 激活函数选择，使用非线性激活函数，如ReLU（Rectified Linear Unit）或Leaky ReLU，可以帮助减轻梯度消失问题。2) 权重初始化，合适的权重初始化可以帮助避免梯度消失。例如，使用较小的方差来初始化权重，可以保持梯度的合理大小。3) 批归一化（Batch Normalization），是一种在每个小批量数据上进行归一化的技术，有助于稳定梯度并加速网络训练。4) 残差连接（Residual Connections），是一种跳跃连接的技术，允许激活和梯度在网络中直接传播。5) 梯度裁剪（Gradient Clipping），是一种限制梯度大小的技术，通过设置一个梯度阈值，可以防止梯度爆炸，并在一定程度上减轻梯度消失问题等等。这些方法可以单独或结合使用，以帮助解决梯度消失问题，并促使了深度学习的爆发。

2，层和块的堆积

为了实现这些复杂的网络，一般引入了神经网络块的概念。块(block)可以描述单个层、由多个层组成的组件或整个模型本身。使用块进行抽象的一个好处是可以将一些块组合成更大的组件，这一过程通常是递归的。通过定义代码来按需生成任意复杂度的块，我们可以通过简洁的代码实现复杂的神经网络。



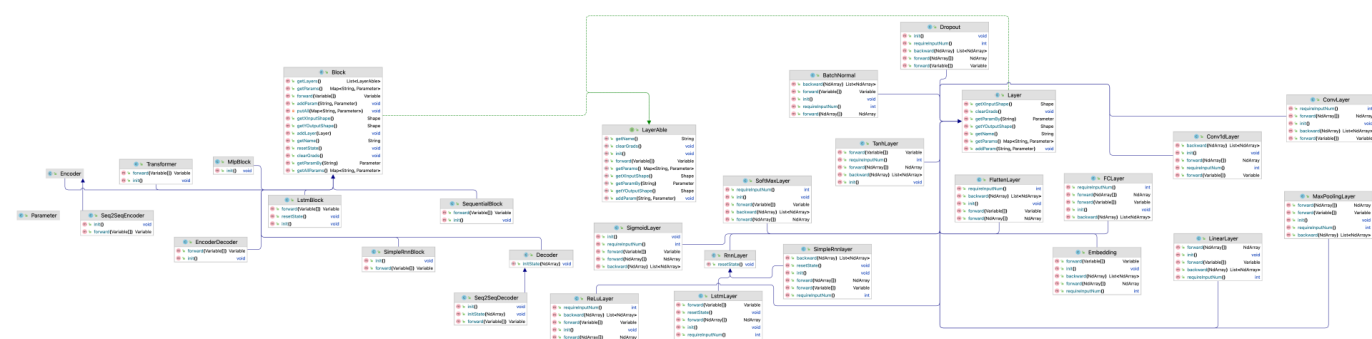
```

1  public interface LayerAble {
2
3      String getName();
4
5      Shape getXInputShape();
6
7      Shape getYOutputShape();
8
9      void init();
10
11     Variable forward(Variable... inputs);
12
13     Map<String, Parameter> getParams();
14
15     void addParam(String paramName, Parameter value);
16
17     Parameter getParamBy(String paramName);
18
19     void clearGrads();
20
21 }
22
23 /**
24  * 表示由层组合起来的更大的神经网络的块
25  */
26 public abstract class Block implements LayerAble
27
28 /**
29  * 表示神经网络中具体的层
30  */
31 public abstract class Layer extends Function implements LayerAble

```

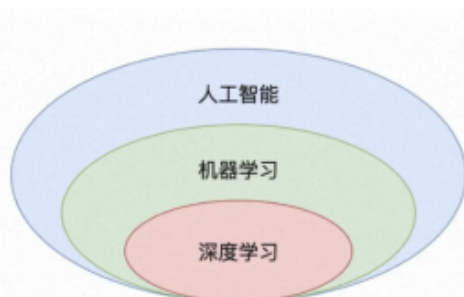
下图是nnet层的整体类图，核心是围绕着Layer以及Block的实现，其中Block是Layer的容器类时 目录与文件的关系，其他都是围绕着它们的实现，每一种Layer或Block的实现，都是一篇知名的学术论文，其

背后都有很深的数学推导（解释为什么网络中加入了该类型的层是有效的）：



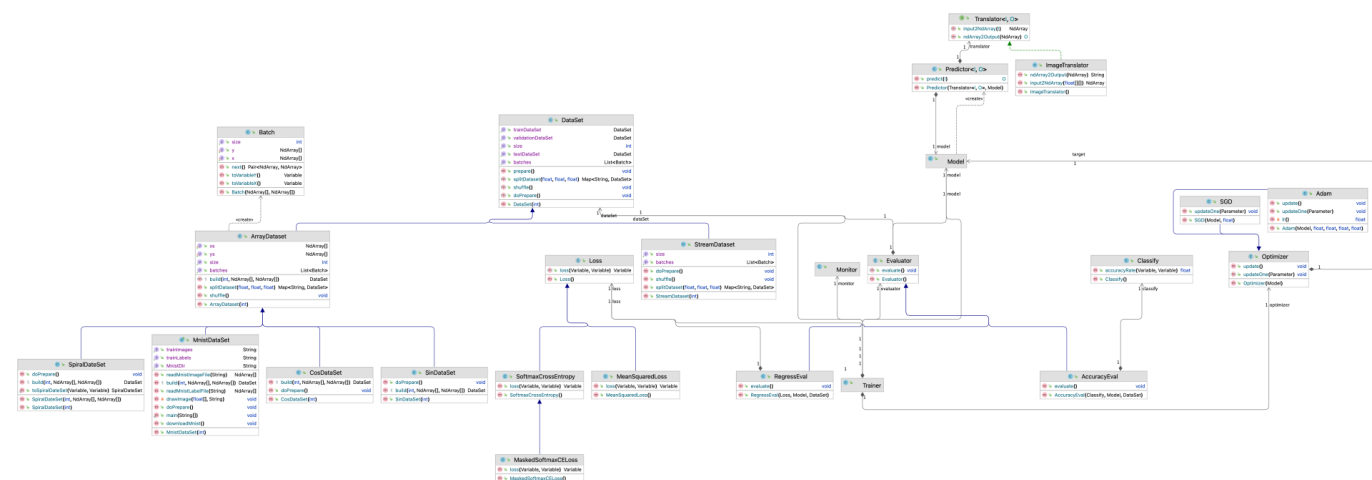
五，机器学习与模型

先梳理下机器学习与深度学习的关系，如下深度学习只是神经网络向纵深发展的一个分支，深度学习只是机器学习的一个分支，一种模型的特例。



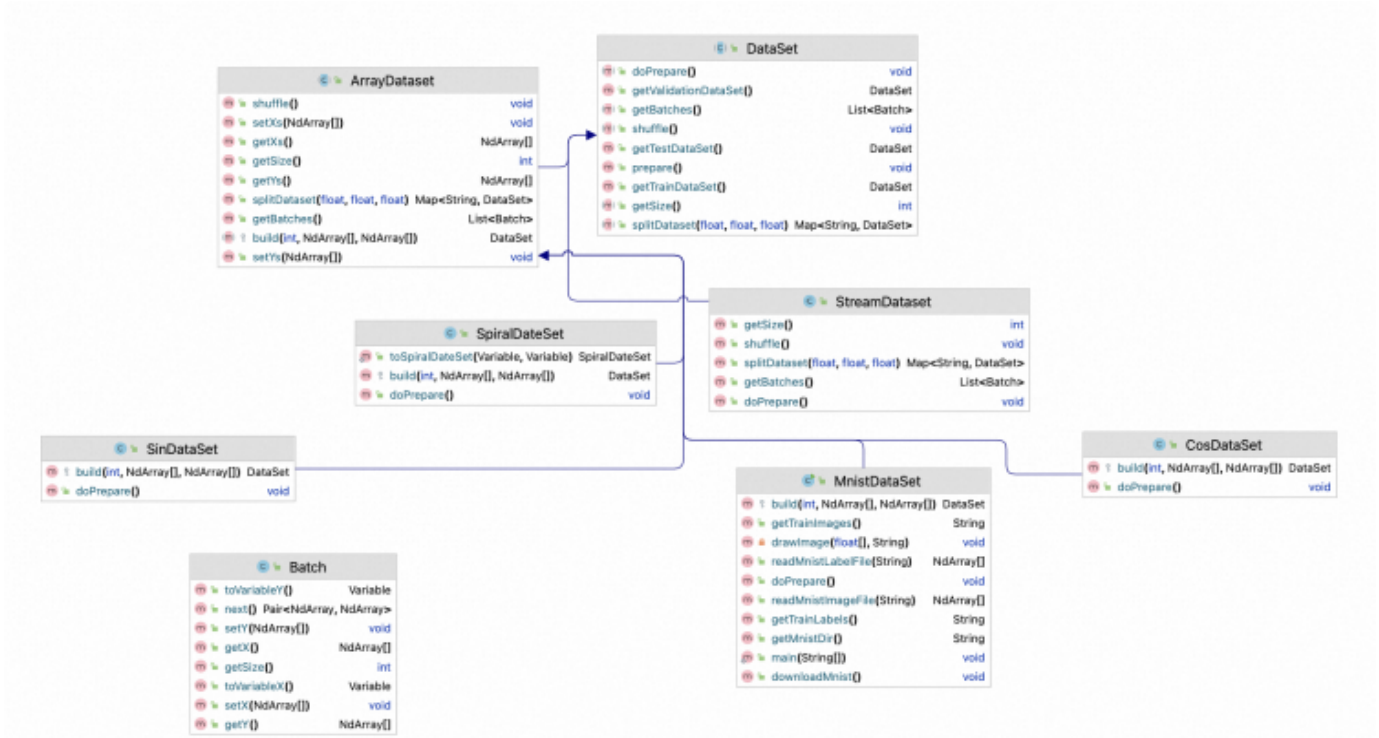
对应更广泛的机器学习则有一套通用的组件，包括数据集损失函数，优化算法，训练器，推导器，效果评估器等。在tinyDL中，讲机器学习的通用组件于深度学习没有强绑定在一起，作为单独的一层来实现，也便于后续扩展出更多非神经网络的模型，例如随机森林，支持向量机等。

如下图所示，是mlearning的整体实现：



1，数据集

DataSet组件定位为数据的加载和预处理转化成模型可以学习的数据格式。目前一些简单数据源实现，都是基于数据可以一次装载到内存的实现（ArrayDataset），例如SpiralDataSet以及MnistDataSet等。

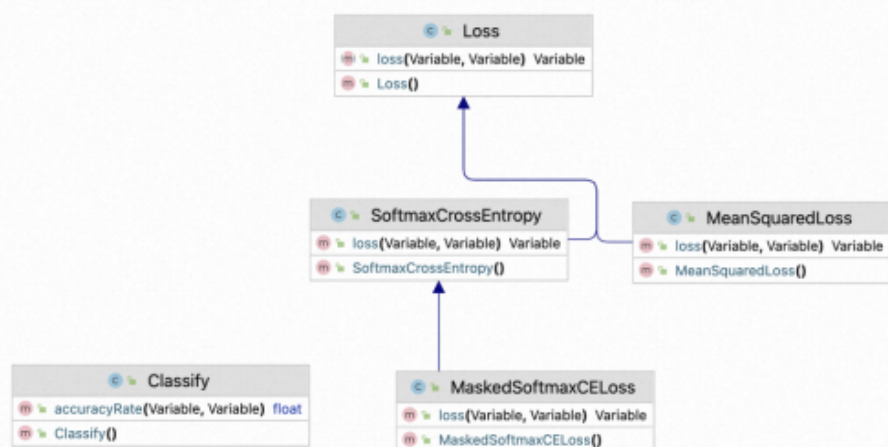


2，损失函数

Loss函数（损失函数）用于度量模型预测值与实际值之间的差异，或者说模型的预测误差。它是模型优化的目标函数，通过最小化损失函数来使模型的预测结果更接近实际值。

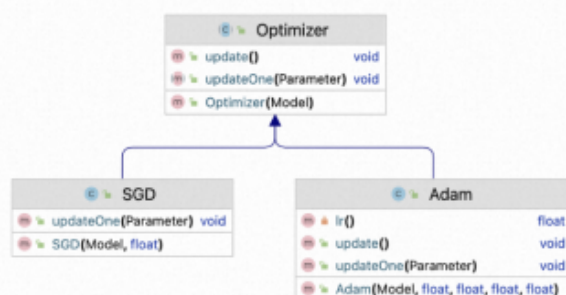
损失函数的选择取决于所解决的问题类型，例如分类问题、回归问题或者其他任务。常见的损失函数有：1) 均方误差（Mean Squared Error, MSE）：用于回归问题，在预测值和真实值之间计算平方差的均值。2) 交叉熵（Cross Entropy）：用于分类问题，比较预测类别的概率分布与真实类别的分布之间的差异。3) 对数损失（Log Loss）：也用于分类问题，基于对数似然原理，度量二分类模型的预测概率与真实标签之间的差异等等。损失函数的选择应该与模型任务和特点相匹配，合适的损失函数能够提供更好的模型性能和训练效果。

目前tinyDL实现了最常用的MSE和softmaxCrossEntropy，其中SoftmaxCrossEntropy将回归问题转化为分类问题的方法是将回归输出结果映射为类别的概率分布。通过SoftmaxCrossEntropy，回归问题可以被转化为多分类问题，模型可以通过最小化SoftmaxCrossEntropy损失函数来进行训练和优化。预测阶段，可以根据概率分布选择概率最大的类别作为预测结果。



3，优化器

机器学习中常用的优化器常用的有：1) 随机梯度下降法（SGD），每次只使用一个样本来计算梯度和更新参数，相比于梯度下降法，计算速度更快。2) 批量梯度下降法，每次使用整个训练集来计算梯度和更新参数，收敛速度相对较慢，但稳定性较好。3) 动量优化器，通过引入动量项来加速梯度下降的更新过程，可以在参数更新时考虑之前的梯度变化情况，从而减少震荡。4) AdaGrad，根据参数的历史梯度进行学习率的自适应调整，对于频繁出现的特征会降低学习率，对于稀疏出现的特征会增加学习率。5) Adam，结合了动量优化器和RMSProp的优点的优化器，Adam优化器通常具有较快的收敛速度和较好的性能等等。目前tinyDL实现了最常用的SGD和Adam：

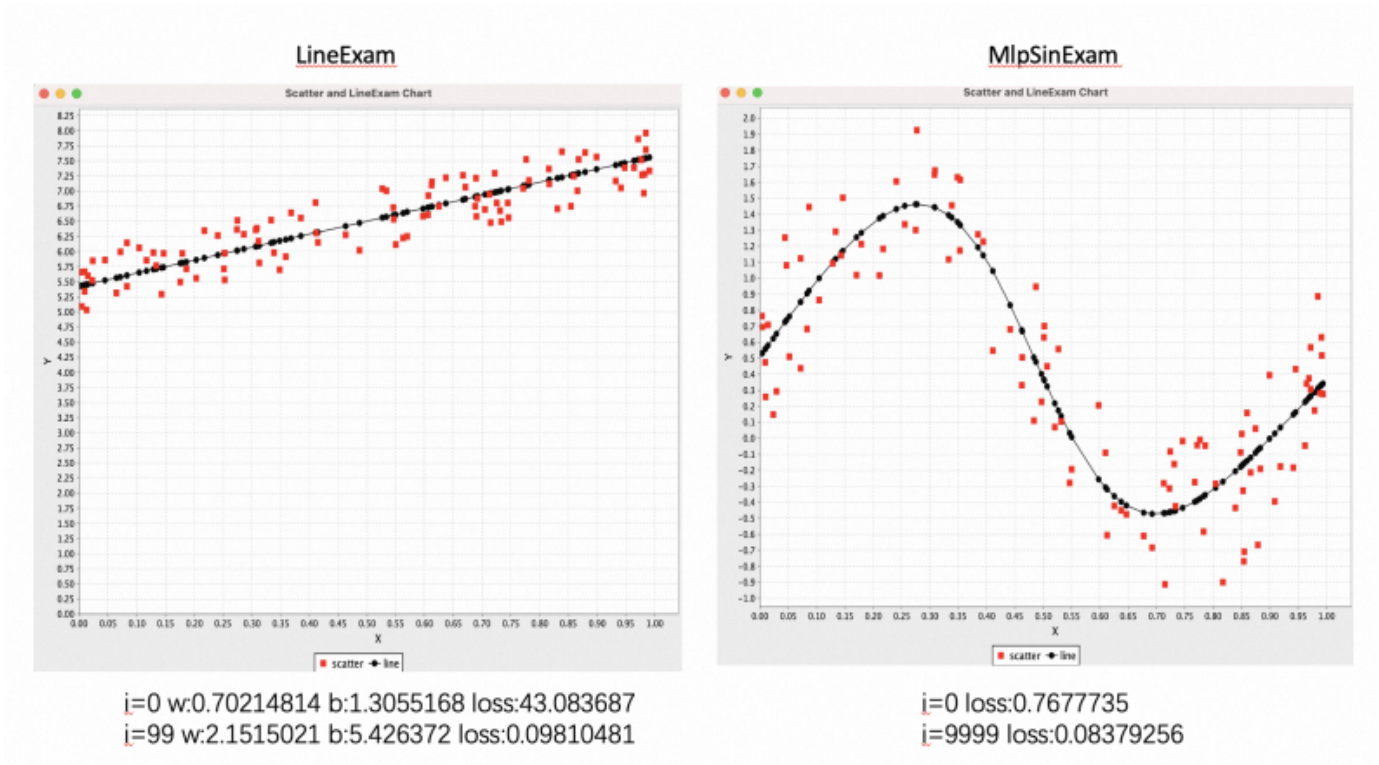


六，应用任务与小试

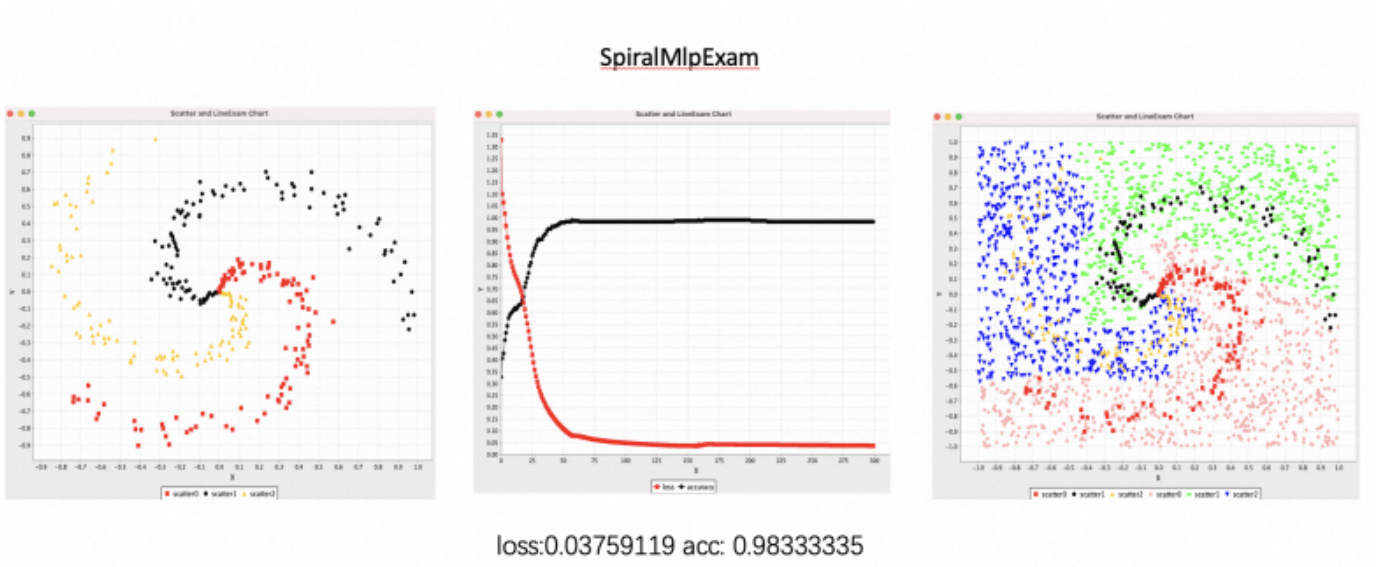
深度学习可以在许多领域中应用，特别在计算机视觉，例如深度学习在图像分类、目标检测、物体识别、人脸识别、图像生成等方面有广泛应用，还有自然语言处理，应用于机器翻译、文本分类、情感分析、语义理解、问答系统等领域，如智能助理、社交媒体分析等。其中目前的大模型的AIGC的两个主要的方向分别是 text2Image以及 text2text，主要的模型架构是stblediffusion与 transformer。这些是依

赖于强大完善的基础能力建设的，目前tinyDL-0.01目前还只能支持一些小模型的训练和推导。如下：

1，直线和曲线的拟合



2，螺旋数据分类问题



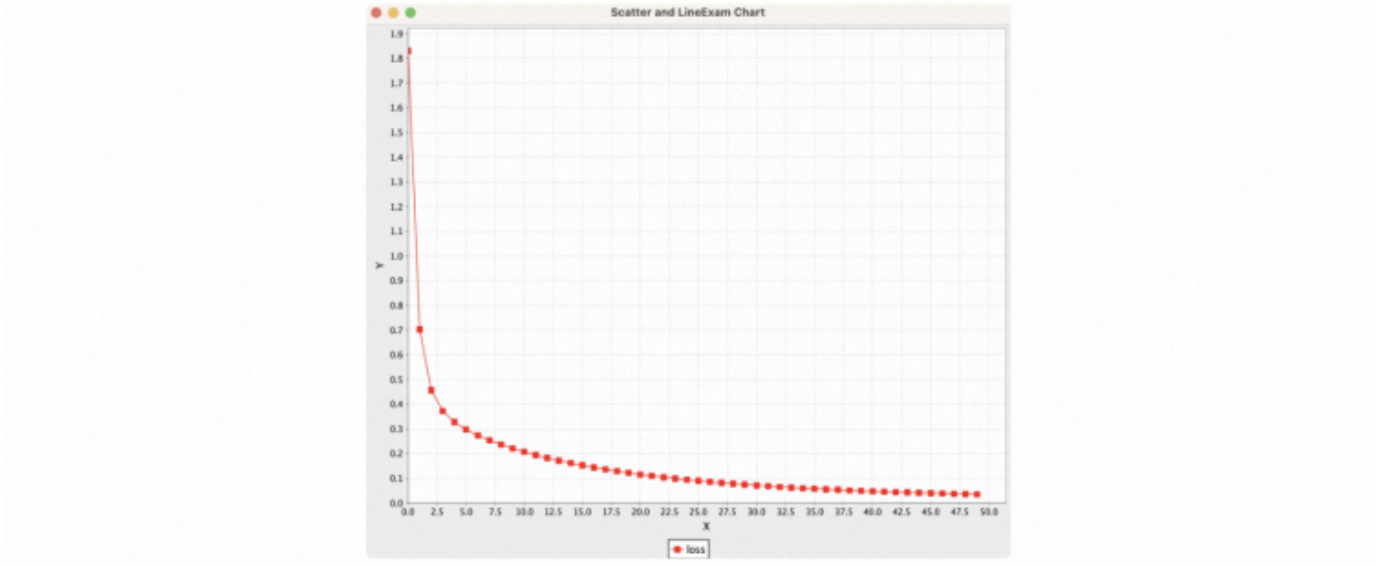
其中第三张图，可以看到模型可以学习出非常清晰的区块边界。

3，手写数字分类问题



在深度学习中，手写数字分类问题是一个经典的问题，常用于介绍和学习深度学习算法，该问题的目标是将手写的数字图像正确地分类成相应的数字。

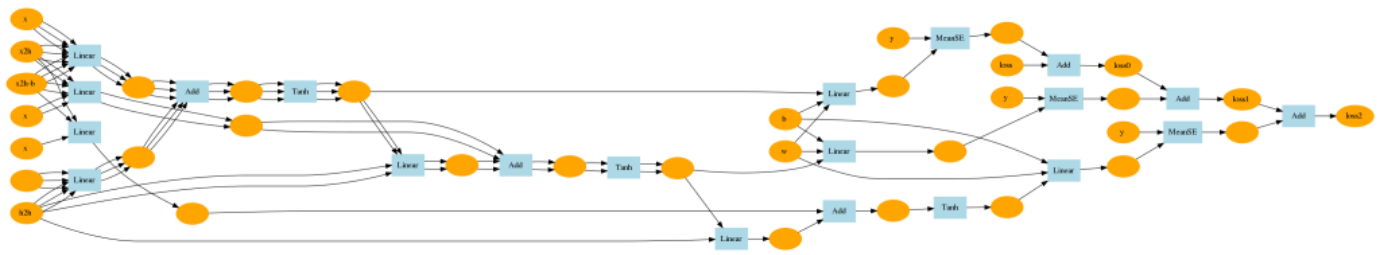
简单地经过50轮的训练后，loss从1.830 减少到0.034。在测试数据集上，预测的准确达到96%(世界上最好的准确度应该做到了99.8%)。



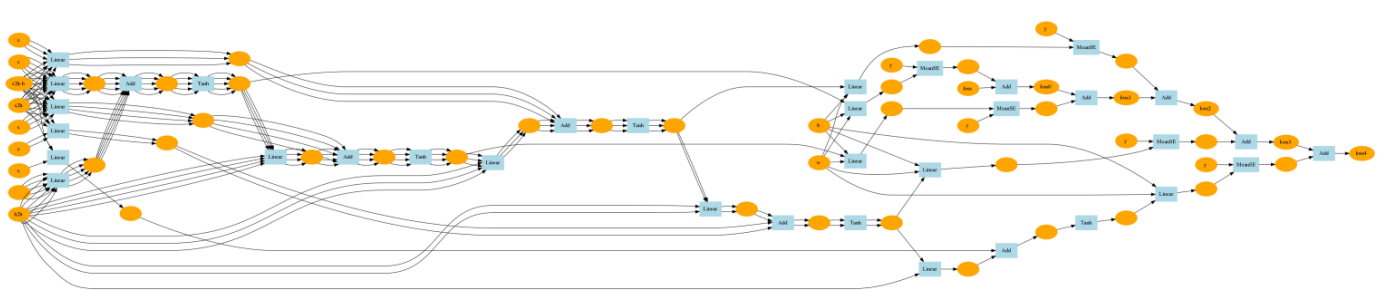
4，通过RNN拟合cos曲线

rnn的计算图如下：

递归隐藏层窗口为3的时候计算图：



递归隐藏层窗口为5的时候计算图：



七，小结

1，TinyDL只是一个对Javaer入门AI友好的Demo框架

一直坚信整洁的代码会自己说话，争取做到代码本身文档。以上写了那么多，其实最好的方式是直接debug一下。同时TinyDL只是为了对Javaer学习AI的一个Demo级别的深度学习框架，目前其并不适合真实环境的使用，但是希望对Java程序员拥抱AI起到一点点作用。就目前的趋势看，Python在AI领域的生态优势已经非常明显了，且Python最开始就是对数学表达友好的语言，其在运算符上有大量的重载数学可读性更强。所以要想真的把AI的能力用起来，Python应该是绕不过的坎。

2，TinyDL-0.02 计划把todo的能力补上，支持一些高级网络特性

其中最近大火的Transformer网络架构和attention层，由于时间匆忙都没来得及实现，TinyDL-0.02希望基于seq2seq的框架实现 tansfromer的原型；还有模型训练上，目前是最简单的单线程运行的，后续会实现参数服务器的分布式网络训练的原型。

3，利用chatGPT协助写代码真的可以提升效率

TinyDL中大概1/3的代码是借助chatGPT完成的，用最近比较流行的说法：程序员将成为自己最后的掘墓人，chatGPT之后码农命运的齿轮开始反转。



SoftmaxCrossEntropy的Java实现，包括forward和backward函数 不要依赖其他二三方库



ChatGPT3.5(16K)

```
1 public class SoftmaxCrossEntropy {
2     private double[][] predictions;
3     private int[] labels;
4     private double[][] gradients;
5
6     public SoftmaxCrossEntropy(double[][] predictions, int[] labels) {
7         this.predictions = predictions;
8         this.labels = labels;
9     }
10
11     public double forward() {
12         double loss = 0.0;
13         int numSamples = predictions.length;
14         int numClasses = predictions[0].length;
15
16         for (int i = 0; i < numSamples; i++) {
17             double[] softmax = softmax(predictions[i]);
18             for (int j = 0; j < numClasses; j++) {
19                 if (labels[i] == j) {
```

🔄 再来一次

🗑️ 清除记忆